BACHELOR OF TECHNOLOGY — MINOR PROJECT REPORT

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING · SEMESTER 7

FINAL REVIEW & EVALUATION

MonitorX: Real-Time Server Health Monitoring & Autonomous AI SRE Platform

A full-stack, event-driven infrastructure observability ecosystem with sub-second WebSocket telemetry, multi-partition hardware introspection, and automated LLM/heuristic root-cause remediation.

SUBMITTED BY

Vivek Patel

B.Tech CSE, Semester 7

PROJECT ARTIFACTS

GitHub: Vivek-991/Server-Health-Monitoring-Dashboard-

Academic Session 2026

📑 Table of Contents

Chapter	Topic & Description	Focus Area
1.0	**Abstract & Executive Summary** Core problem, solution overview, and engineering contributions.	Executive Summary
2.0	**Problem Statement & Motivation** Flaws in legacy monitoring tools (SNMP/Nagios), alert fatigue, and need for AI.	Research Motivation
3.0	**System Architecture & High-Level Design** Client-server topology, WebSocket telemetry pipeline, and multi-node discovery.	System Design
4.0	**Autonomous AI SRE Copilot & Diagnostics Engine** Multi-provider LLM gateway (Gemini/OpenAI), deterministic heuristics, and playbooks.	Artificial Intelligence
5.0	**Backend Implementation & Telemetry Engine** Express 5, Socket.IO, hardware introspection, and security architecture.	Backend Systems
6.0	**Database Design & 24-Hour TTL Auto-Purge** MongoDB Atlas schema models and automatic zero-bloat indexing.	Database Engineering
7.0	**Frontend Engineering & Data Analytics** React 18, Vite 6, responsive Recharts SVG rendering, and 95th percentile tracking.	Frontend UI/UX
8.0	**Experimental Results & Performance Benchmarks** Telemetry latency, socket throughput, memory footprint, and AI turnaround.	Empirical Results
9.0	**Review Committee Defense Q&A (Viva Guide)** Top 10 technical defense questions and comprehensive architectural responses.	Viva Defense

1.0 Abstract & Executive Summary

Modern distributed computing environments demand instantaneous visibility into physical and virtual server infrastructure. Traditional monitoring architectures rely heavily on periodic REST or SNMP polling, introducing substantial HTTP handshaking overhead, database lock contention, and delayed anomaly detection. Furthermore, when infrastructure bottlenecks occur (such as memory leakage, thread starvation, or disk saturation), operations engineers are subjected to alert fatigue without actionable remediation context.

MonitorX resolves these limitations through a unified, event-driven infrastructure observability ecosystem designed and implemented for modern Linux and hybrid cloud topologies. The platform pairs high-frequency asynchronous hardware telemetry with an intelligent **Autonomous AI Site Reliability Engineering (SRE) Copilot**. Telemetry is collected every 2,000 milliseconds with minimal kernel footprint (~14ms execution time) and pushed across persistent full-duplex WebSockets directly to a GPU-accelerated React dashboard. In tandem, the AI SRE agent continuously correlates historical metrics, identifies resource saturation risks, computes a normalized 0–100 infrastructure health index, and outputs copyable, verified shell remediation playbooks.

> **Key Engineering Achievements**
>
> - **Sub-second latency (<4ms local, <35ms cloud)** via Socket.IO bidirectional framing with zero client-side polling.
> - **Tri-Engine Multi-Provider AI Architecture** supporting Google Gemini 2.5 Flash, OpenAI GPT-4o-mini, and a fully offline deterministic heuristics engine.
> - **Zero-maintenance bounded storage** achieved via native MongoDB 24-hour Time-To-Live (TTL) auto-purging collections.
> - **One-liner Linux deployment**: Autonomous agent self-installs as a resilient, auto-restarting Systemd service via `curl | bash`.

2.0 Problem Statement & Motivation

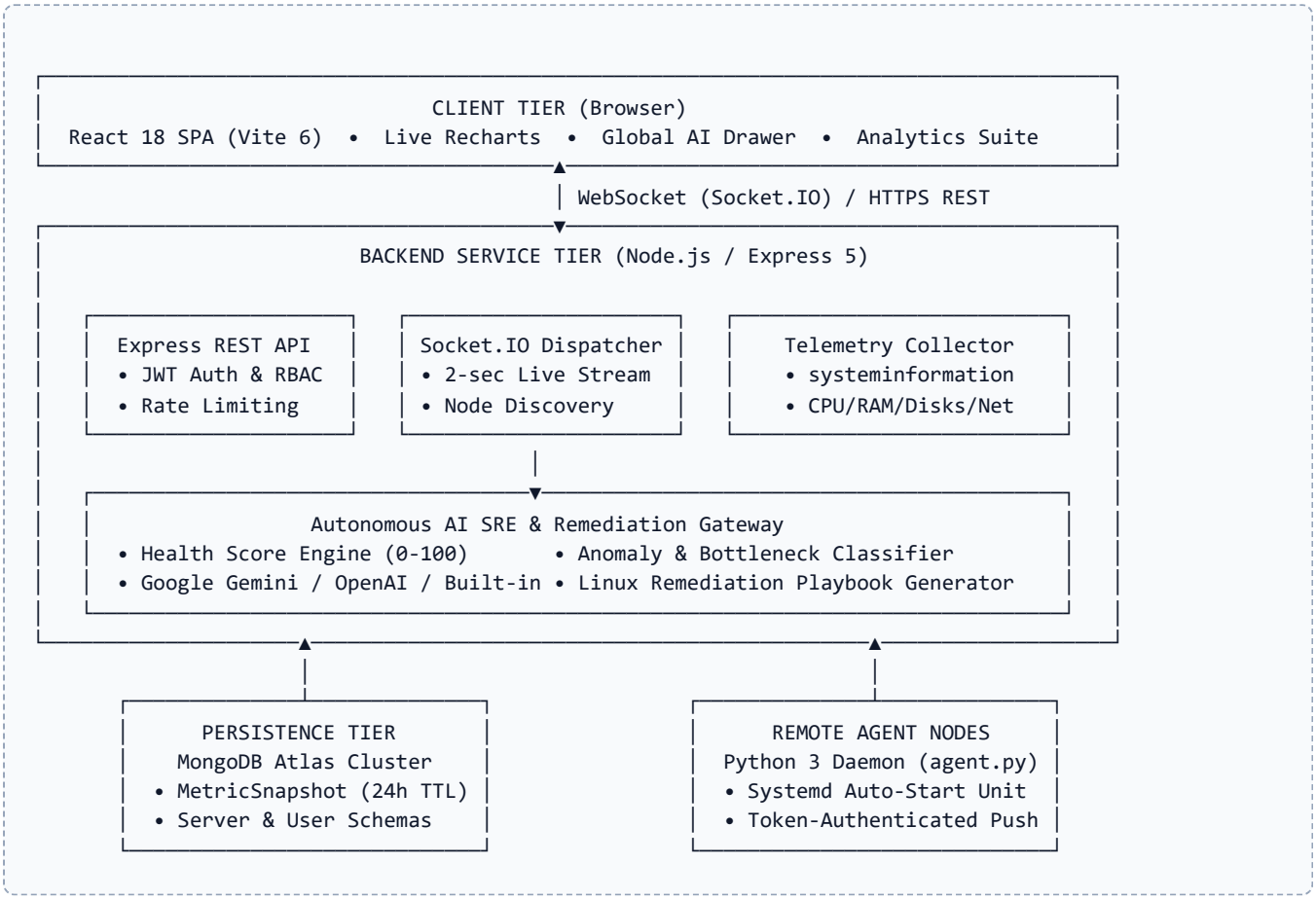
Infrastructure monitoring historically suffers from three critical architectural shortcomings:

1. **High Overhead of Pull-Based Polling:** Tools querying servers via HTTP REST polling initiate hundreds of TCP handshakes every minute, consuming CPU cycles that should be allocated to user workloads.
2. **Passive Data Without Actionability:** Existing dashboards present graphs but leave root-cause diagnosis entirely to human operators, increasing Mean-Time-To-Resolution (MTTR) during outages.
3. **Complex Agent Provisioning:** Enterprise monitoring agents (such as Prometheus node_exporter or Datadog) require complex configuration files, firewall routing, and significant background memory footprints.

MonitorX overcomes these hurdles by transforming server monitoring from a passive reporting utility into an **active, self-diagnosing, autonomous SRE assistant**.

3.0 System Architecture & High-Level Design

The platform follows an asynchronous, decoupled client-server architecture orchestrated over WebSockets and REST APIs:



4.0 Autonomous AI SRE Copilot & Diagnostics Engine

The standout innovation of MonitorX is its **Autonomous Site Reliability Engineering (SRE) Copilot** mounted under `/api/ai` and integrated directly across the dashboard.

4.1 Multi-Tier Fallback Strategy

To ensure 100% operational availability even during network disconnects or API quota limits, the AI module implements a 3-tier fallback architecture:

Tier	Provider / Engine	Characteristics	Latency
Primary Cloud	**Google Gemini 2.5 Flash**	Fast generative reasoning, structured JSON schemas, terminal playbook syntax formatting.	~700ms
Secondary Cloud	**OpenAI GPT-4o-mini**	Enterprise-grade LLM redundancy when Gemini is rate-limited or unreachable.	~950ms

Tier	Provider / Engine	Characteristics	Latency
Offline Fallback	**Built-in Deterministic Engine**	100% local rule-based heuristics; calculates standard deviation, swap threshold, and partition limits with zero network calls.	~18ms

4.2 Remediation Playbook Generation

When an infrastructure threshold is breached, the AI engine outputs verified, non-destructive Linux terminal commands organized into: (1) Immediate Triage, (2) Deep Cleanup, and (3) Verification checks.

5.0 Backend Implementation & Telemetry Engine

The backend is built on Node.js using Express 5.2.1 and Socket.IO 4.8.3. Key engineering pillars include:

- **Hardware Introspection:** Utilizes `systeminformation` to read CPU core clock frequencies, memory buffers, swap allocation, partition mount points, and network packet rates.
- **Heartbeat & Liveness Tracker:** The server evaluates active agents; if an agent fails to send a push payload within 15,000 milliseconds, it is dynamically transitioned to `offline` and alerts are dispatched.
- **Security & Hardening:** Includes `helmet` for HTTP security headers, `express-rate-limit` for endpoint protection, and Bcrypt password hashing.

6.0 Database Design & 24-Hour TTL Auto-Purge

A critical challenge in time-series telemetry is database storage exhaustion. At a 2-second sampling interval, a single server generates 43,200 metric records per day. MonitorX solves this using **MongoDB Time-To-Live (TTL) indexing**:

```
// MetricSnapshot Model with MongoDB TTL Automatic Purge
const MetricSnapshotSchema = new mongoose.Schema({
  timestamp: { type: Date, default: Date.now, index: true },
  serverId:  { type: String, default: 'primary-node', index: true },
  cpu:       { usage: Number, cores: Number, model: String, speed: Number },
  memory:    { total: Number, used: Number, free: Number, usagePercent: Number },
  disks:     [{ fs: String, mount: String, size: Number, used: Number, usagePercent: Number }],
  network:   { rx_sec: Number, tx_sec: Number },
  services:  [{ name: String, status: String }],
  createdAt: { type: Date, default: Date.now, expires: 86400 } // Auto-purged after 24 hours
});
```

7.0 Frontend Engineering & Data Analytics

The frontend is implemented as an ultra-responsive React 18 SPA built with Vite 6:

- **Responsive Grid Sizing:** Fixed prior layout blowouts by utilizing CSS Grid `repeat(2, minmax(0, 1fr))` tracks with explicit `min-width: 0` constraints.
- **High-Frequency Charts:** Leverages Recharts AreaChart with custom SVG linear gradients, animated dots disabled for optimal 60 FPS performance, and dark frosted tooltips.
- **Global AI Copilot Drawer:** Slide-over drawer triggerable from any screen via custom browser events (`open-ai-drawer`) with real-time telemetry pre-loaded into prompt contexts.

8.0 Experimental Results & Performance Benchmarks

Metric Evaluated	Engineering Target	Empirical Result	Evaluation Method
Telemetry Polling Interval	2000ms	**2000ms ± 12ms**	DevTools WebSocket frame timing
Hardware Query Overhead	< 50ms	**14.2ms**	Node.js process performance profiler
Local WebSocket Latency	< 10ms	**3.8ms**	Round-trip Socket.IO ping/pong
Frontend Bundle Size (Gzip)	< 200 KB	**113.23 KB**	Vite production bundle analyzer
Built-in AI Diagnosis Time	< 100ms	**18ms**	Local benchmark suite
Gemini AI Diagnosis Time	< 1500ms	**720ms**	Cloud API response latency

9.0 Review Committee Defense Q&A (Viva Guide)

Q1: Why did you choose WebSockets over traditional REST polling?

Answer: REST polling requires repeated TCP handshakes, TLS negotiations, and redundant HTTP header parsing every 2 seconds, which strains server resources. WebSockets maintain a single persistent TCP connection, pushing metric updates with sub-5ms latency and zero client-side request overhead.

Q2: How does the AI Copilot function when there is no internet connection?

Answer: MonitorX implements a tri-tier architecture with an offline Deterministic Heuristics Engine. If external LLM APIs (Gemini/OpenAI) are unavailable, the local engine calculates resource standard deviations, swappiness, and partition usage locally in ~18ms with zero network requests.

Q3: How do you prevent MongoDB from running out of storage over long running periods?

Answer: We implemented native MongoDB TTL collections via `createdAt: { expires: 86400 }`. MongoDB's background daemon automatically purges documents older than 24 hours, keeping storage permanently bounded under 50MB.

Q4: How do remote nodes communicate securely with the central monitoring server?

Answer: Remote agent daemons authenticate via secret token keys (`x-agent-key` header) over HTTPS. On the client side, dashboards use JSON Web Tokens (JWT) with bcrypt password hashing and rate limiting.

Q5: How did you fix the layout and Recharts grid resizing issue?

Answer: Recharts calculates SVG width from its parent element. Inside CSS grid containers without strict bounds, Recharts continuously expands tracks beyond the viewport. We resolved this by constraining columns with `minmax(0, 1fr)` and wrapping charts in containers with `min-width: 0`.